AI For Accounting Games

Developer Manual

By

Sarah Sessions

Helen Grogan

Cole Suchan

Edward Calinsky

Jayden LaCross

Submitted in partial fulfillment of the requirements for

COMP 4710 Senior Design

to the Department of Computer Science and Software Engineering,

Samuel Ginn College of Engineering, Auburn University

Auburn, Alabama

Table of Contents

Project Introduction **3**

What is AI for Accounting Games? 3

Where did you leave off? 4

What should future developers focus on? 4

Points of Contact 5

Project Details **6**

Playing the Game Online 6

GitHub Repository 6

Project Infrastructure 6

Getting Started **8**

Required Packages 8

Cloning the GitHub Repository 10

Repository Navigation 11

Running a Local Copy of the Game 11

Development Cycles **15**

Local Changes 15

Updating the Server 15

Other Tips 18

Additional Information **20**

SSL/HTTPS 20

SAML Authentication 20

AD Group Parser 23

Database 24

API Endpoints 25

Project Introduction

What is AI for Accounting Games?

AI for Accounting Games is an ongoing collaboration with the Harbert College of Business and is being sponsored by Professor Andrew McLelland. The game is intended to aid students, specifically those in ACCT 2110 and 5110, to build and improve their understanding of the crucial accounting principles introduced and taught in these courses. Extracting data from game results allows professors to track their students' progress, further enabling them to help their students in areas they may still be struggling with.

As it stands now, *Accounting Game* prompts users to sign in with their Auburn credentials using AUthenticate, in a process known as Single Sign On (SSO). After logging in, user data from SSO is processed to determine whether the user is an administrator, professor, or a student, and which classes for Accounting they may be enrolled in, if any. After playing the games, the user's progress is tracked on the leaderboard. Professors and Administrators can look at these leaderboards on their respective dashboards, which include other data such as average score and time played. They may also download the data if they so choose. This data can be used to track student progress in the game, and to discover if the student may need help in certain areas of the course. AI applications may also be made to this data to find patterns in students' scoring behavior.

Accounting Game currently includes two game modes with five total games that cover basic accounting principles: *Debit vs. Credit*, *The Five Elements*, and *Accounting Equation*. *Debit vs. Credit* has users sort different accounting terms based on if they are a credit or a debit. *The Five Elements* is a similar game to *Debit vs. Credit*, where students sort accounting terms into the five main accounting elements (assets, expenses, liabilities, shareholder equity, and revenues). *Accounting Equation* is a quiz game that has students answer questions using the primary accounting equation (Assets = Liabilities + Shareholder Equity). The first two games function using the WASD or arrow keys to move a sprite that can pick up accounting terms and place them on a conveyer belt that directs it into the proper category. The space bar is utilized for

picking up and placing down terms. The mouse and mouse clicks can also be used for controlling the sprite. *Accounting Equation* utilizes the mouse to select answer choices or equation values after the question is displayed.

https://accounting-game.cse.eng.auburn.edu/

Where did you leave off?

Professor McLelland tasked us with two big jobs: OIT integration with SSO, and a user database to track in-game performance. With those goals came several challenges, but these objectives have been completed. *Accounting Game* now asks students, professors, and administrators to log in with AUthenticate when launching the game, and can accurately track stats, display names on the leaderboards, and show professors information, all without any hassle for the user. This creates a seamless gameplay and analytical experience for all of our clients at Auburn University, making A*ccounting Game* an extremely versatile edutainment tool for the Harbert College of Business. Along with our main focuses, several other improvements have been made to the game, such as visual improvements, enhanced feedback, customizable speed, settings menu, and admin/professor dashboards for student data.

What should future developers focus on?

Professor McLelland has expressed interest in presenting *Accounting Game* to others in the business education field, as well as potentially launching this game in other universities. Future improvements may also include visual enhancements, such as upscaling the resolution or updating textures, gameplay improvements such as adding games or enhancing gameplay, and

structural improvements, such as improving backend workflow and implementation with our virtual machine.

If this game is indeed taken to other universities, consideration will also have to be made as towards the hosting of the game. Currently, the game is hosted on an Auburn University machine, only accessible through AU Wi-Fi. To make this game playable for other students, host machines at other universities would need to be instated, or our host would have to be given the ability to connect to students at other campuses. Server demand and web traffic would also have to be considered.

Points of Contact

Sponsor | Andy McLelland | mclelaj@auburn.edu

Server Host | Kelly Price | pricekg@auburn.edu

SSO Administrator | Jason Buckman | jpb0029@auburn.edu

Project Details

Playing the Game Online

Accounting Game is hosted on an Auburn University virtual machine running Linux provided by Computer Science Faculty **Kelly Price**. To access the game online, type the host name **accounting-game.cse.eng.auburn.edu** into your web browser. This game is only accessible when connected to eduroam using on-campus WiFi. To connect to the game from home, use the **GlobalProtect VPN** provided by the University. An AU Windows machine from **https://rdp.eng.auburn.edu/** may also be used to connect to the website in the event the VPN does not work.

GitHub Repository

Our source code can be found at the following GitHub repository:

https://github.com/EdwardACaly/Accounting-Game

Project Infrastructure

The project consists of three major components:

The Web Browser Application *(the **frontend**)*

This webapp was built using the **Phaser Framework. npm** is used to run the **Vite** scripts for development and deployment builds. The game is currently hosted on an **nginx** webserver, but any webserver may be used. Software development involving user-facing portions of the game, including most of the mechanics involving the game itself, and its scenes, will be referred to as the **frontend**.

The Database and API *(the **backend**)*

The leaderboard, SSO, and admin/professor views make use of **PostgreSQL** for database management. **Python FastAPI** is used for the API providing an interface between the frontend and the database. Anything involving the database, API, or server-side changes will be referred to as the **backend**.

Auburn University Virtual Machine *(the **server**)*

Our game is hosted on a virtual machine within an Auburn University-owned server, with hostname accounting-game.cse.eng.auburn.edu. Any changes to this server must be done through **Secure Shell (SSH)**, and the game can only be accessed online from Auburn University WiFi. To see the live online version from home, use the **GlobalProtect VPN,** or connect to the website from **https://rdp.eng.auburn.edu/**.

Getting Started

NOTE: The following instructions are given for a x64 Windows operating system. Instructions for Mac/Linux may vary.

Required Packages

The following packages are required to be installed **before starting development**:

Python 3.13.2

The current version of Python this project is running on is **version 3.13.2**. Please **ensure you are running this version of Python**, as older/newer versions, including the version downloaded from the Windows Store, may cause issues when working on different aspects of the project.

To install: go to **https://www.python.org/downloads/** and select **Python 3.13.2** from the Specific Releases list further down the page. Once installed, make sure you add

`C:\Users\<USER>\AppData\Local\Programs\Python\Python313\Scripts\` and `C:\Users\<USER>\AppData\Local\Programs\Python\Python313\` to your PATH variable, at the top of the list (filepath may vary based on install location, what's listed above is usually the default).

git

git is used to download, upload, push, pull, branch, and manage the remote repository during development. We highly encourage solidifying a **branching method** during development so your group can work productively throughout the course of the semester.

To install: go to **https://git-scm.com/install/** and download the most recent version of git for your operating system. The default settings in the Git Setup Wizard are fine for this project.

Node.js

This webapp is built primarily using JavaScript. Node.js is used to run JavaScript code outside of a web browser. **npm (node package manager)**, which is needed for building functionality with Vite, also comes with Node.js.

To install: go to **https://nodejs.org/en/download** and download the **prebuilt Node.js** for your operating system. The default settings in the Node.js Setup Wizard are fine for this project.

nginx

nginx is the webserver used to run the project.

To install: go to **https://nginx.org/en/download.html** and download the **mainline** version of nginx for your operating system. Once the **.zip** file is downloaded, move it to the **root directory** of your PC and extract it there. Once extracted, the zipped folder can be deleted. Open the nginx folder in the terminal, and run `start nginx`.

PostgreSQL

PostgreSQL (also known as postgres) is the software used for database creation and management throughout the project. It works like a command line version of something like DB Browser for SQLite.

To install: go to **https://www.postgresql.org/download/** and download the latest version of PostgreSQL for your operating system. The default settings in the PostgreSQL Setup Wizard are fine for this project. After installation, make sure to add `C:\Program Files\PostgreSQL\18\bin` to your PATH variable, on the last line.

NOTE: During setup, you will be asked to create a password for **postgres**, the database superuser. This is essentially like a **sudo** password in Linux, only used for the database specifically. Remember the password you create here, as you will need it when working with the database later.

After installing all required packages, make sure to **restart any terminal windows** you may have opened, to ensure all packages are correctly installed and initialized.

Cloning the GitHub Repository

Before starting work on this project, we recommend **creating your own GitHub repository** using the code from ours. After doing so, you may clone, push, and pull request seamlessly from your own remote repository. Once your repository is created, follow the steps below to begin working on your implementation of *Accounting Game*.

Navigate to an empty project directory to use for development.

On your remote repository's GitHub page, find the **HTTPS link** for your repo under <> Code:

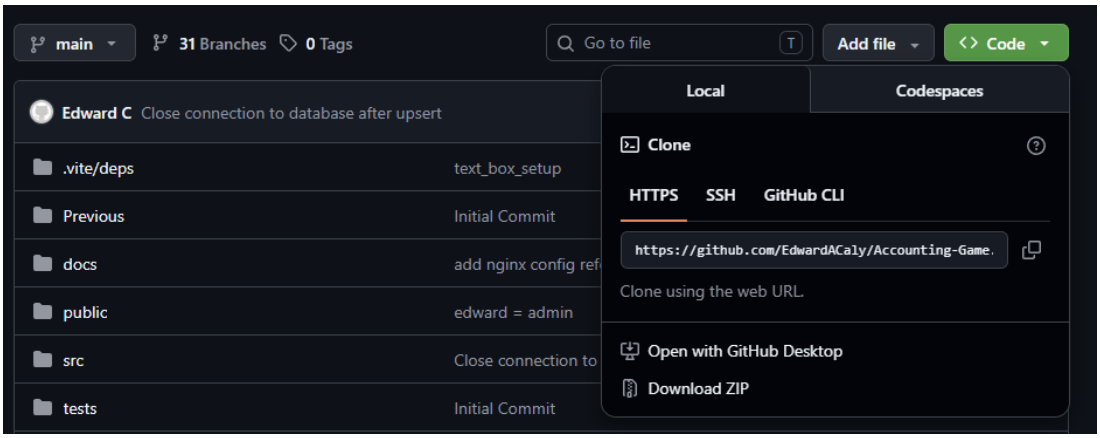

Figure 1.1: Finding the remote repo link on GitHub.

Then, run `git clone <repo link>`.

This will create a local clone of the remote repository on your machine. The root directory will be a folder called Accounting-Game.

Repository Navigation

The repository itself is organized into a couple different folders for different aspects of the game:

/src: contains the scenes, entities, and backend files for the entire game. **main.js** runs the main JavaScript file for the frontend.

/src/gameobjects: contains the JavaScript files for all the entities in the game.

/src/scenes: contains the JavaScript files for all the scenes.

/src/backend: contains the .py file and other files related to leaderboard database management. All backend development will occur in this folder. SAML files are also held here.

/public: contains assets for the game, such as sprites, backgrounds, and other images. Loaded in using **/src/preloader.js**.

Running a Local Copy of the Game

Starting the Backend

NOTE: The .env.example file provides an example template for the .env file required to run the backend. The DATABASE_URL field of this file must be modified to include the correct password for the database. Replace <PASSWORD> with the database password and rename .env.example to .env.

NOTE: git will ignore the .env file during pushes/pulls, as the database password should never be shared publicly. After pulling from main after an update, you must remake the .env file.

NOTE: It is recommended to create a virtual environment when first setting up the backend for reproducibility's sake, and to isolate the multitude of dependencies within `requirements.txt`:

```
cd AccountingGame/src/backend

python3 -m venv venv

source venv/bin/activate
```
—> (rerun this line each time you start up backend)

From **/src/backend**, run `pip install -r requirements.txt`. This will install all required python modules for the backend using the requirements.txt file.

Run `psql -U postgres -h 127.0.0.1 -d postgres -v password=<PASSWORD> -f schema.sql` with the correct password in place. This creates the postgres database.

NOTE: This is where you will need the **superuser postgres password** from earlier.

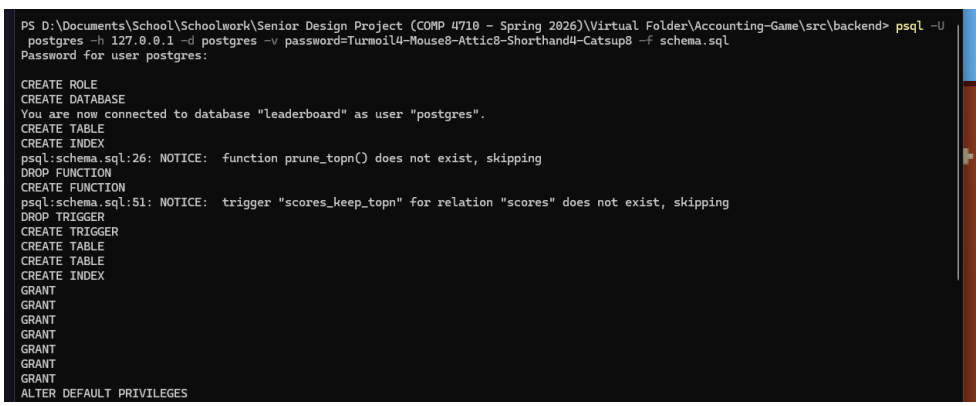

Figure 1.2: Creating the PostgreSQL database.

Run `python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000`. This command starts the backend. From this terminal window, you can monitor the game's requests made to the database during play.

```
PS D:\Documents\School\Schoolwork\Senior Design Project (COMP 4710 - Spring 2026)\Virtual Folder\Accounting-Game\src\backend> python
-m uvicorn main:app --reload --host 0.0.0.0 --port 8000
INFO:     Will watch for changes in these directories: ['D:\\Documents\\School\\Schoolwork\\Senior Design Project (COMP 4710 - Spring
 2026)\\Virtual Folder\\Accounting-Game\\src\\backend']
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [1996] using WatchFiles
INFO:     Started server process [13828]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:62270 - "GET /fetch-user HTTP/1.1" 200 OK
INFO:     127.0.0.1:62270 - "GET /fetch-user HTTP/1.1" 200 OK
INFO:     127.0.0.1:62270 - "GET /fetch-user HTTP/1.1" 200 OK
INFO:     127.0.0.1:62270 - "GET /fetch-user HTTP/1.1" 200 OK
INFO:     127.0.0.1:62270 - "GET /fetch-user HTTP/1.1" 200 OK
```

Figure 1.3: The backend monitors fetches to the database during play.

NOTE: Keep this terminal open to keep the backend running while starting the frontend.

NOTE: The leaderboard data from the server will not appear when running locally. To see a leaderboard on the local side, populate the database using the populate.py script.

NOTE: The backend must be restarted after changes to any .py file in **/src/backend**.

Starting the Frontend

In a separate terminal window, from **/Accounting-Game**, run `npm install` to install all packages for the frontend.

Run `npm run dev` and crtl+click the link to open a local copy the game in your browser.

```
Run "npm audit" for details.
PS D:\Documents\School\Schoolwork\Senior Design Project (COMP 4710 - Spring 2026)\Virtual Folder\Accounting-Game> npm run dev

> template-vite@0.0.0 dev
> vite


  VITE v6.4.1  ready in 280 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Figure 1.4: After launching the game.

NOTE: Keep this window open during play as well. It will **automatically refresh** with new changes if any JavaScript files are modified during runtime.

NOTE: The frontend can also be run **by itself** without the backend if changes to the leaderboard/databases aren't being worked on. If you get leaderboard errors, **make sure your backend is running**!

NOTE: SSO will not work locally when disconnected from Auburn WiFi. When asked to sign on at the start of the game, **click the blue button** and **refresh the page** to reload the game and skip the sign-in screen.

NOTE: To check a raw database table, visit that table's URL while the game is running (e.g. https://localhost:8000/leaderboard/game1 for game 1's database). It will return [] if your local database is empty.

Development Cycles

Local Changes

Pull the most recent version of the remote repo using `git pull origin main`.

After install, you only need `python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000` to start the backend and `npm run dev` to start the frontend.

After committing your changes, use `git push origin <branch name>` to push your branch to the remote repo, at which point you may submit a pull request to merge to main from the GitHub website.

Once all of your major changes have been pushed and merged to main, you may update the server with your changes.

Updating the Server

NOTE: To push changes to the server, you must have access to **sudo privileges** on the virtual machine. Please email Kelly Price and ask for your team to be granted sudo privileges for the server.

Connect to the Server

To connect to the server, run `ssh <auburn username>@gate.eng.auburn.edu`, followed by your password. At the second prompt, for the hostname, enter `accounting-game.cse.eng.auburn.edu`, followed again by your password. After connecting, you should be in `/home/<auburn username>` on the server (Linux environment).

NOTE: Run `cd ..` to access the root directory of the server.

Figure 2.1: Connecting to the game's server.

NOTE: Run `cd ..` to access the root directory of the server.

Push Your Changes

The web build of the game that the server hosts at `accounting-game.cse.eng.auburn.edu` is contained at **/var/www/AccountingGame**. To update this folder, you must build your copy of the game for distribution, and then replace that folder with your distribution.

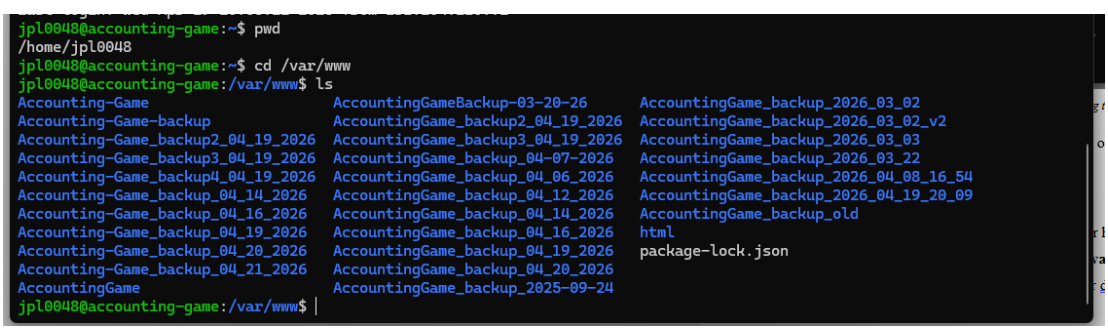

Figure 2.2: The location of AccountingGame at /var/www. This is the folder we will replace.

From your directory on the server, run `sudo git pull <repo URL>` to clone the main branch of the remote repo just as you would from your own computer.

`sudo git clone <repo URL>` can be used but keep in mind this will clone over .env passwords required for the back end which would need to be reinstated.

From **/Accounting-Game**, run `sudo npm run build`. This will create a distribution-ready copy of the game at **/Accounting-Game/dist**.

NOTE: You may need to run `sudo npm install` again here.

```
jpl0048@accounting-game:~/Accounting-Game$ sudo npm install

added 160 packages, and audited 161 packages in 8s

80 packages are looking for funding
  run `npm fund` for details

8 vulnerabilities (4 moderate, 4 high)

To address issues that do not require attention, run:
  npm audit fix

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
jpl0048@accounting-game:~/Accounting-Game$ sudo npm run build

> template-vite@0.0.0 build
> vite build

vite v6.4.1 building for production...
✓ 36 modules transformed.
dist/index.html                     1.07 kB │ gzip:   0.62 kB
dist/assets/index-Dmk8baTw.css      0.21 kB │ gzip:   0.18 kB
dist/assets/index-BYjlpgH1.js     459.55 kB │ gzip: 142.54 kB
dist/assets/phaser-DK13qiGw.js  1,481.80 kB │ gzip: 337.89 kB

(!) Some chunks are larger than 500 kB after minification. Consider:
- Using dynamic import() to code-split the application
- Use build.rollupOptions.output.manualChunks to improve chunking: https://rollupjs.org/configuration-options/#output-manualchunks
- Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.
✓ built in 9.84s
jpl0048@accounting-game:~/Accounting-Game$ ls
LICENSE    deploy_script.sh  index.html         package.json  screenshot.png  tests
Previous   dist              node_modules       populate.py   src             vite.config.js
README.md  docs              package-lock.json  public        style.css
jpl0048@accounting-game:~/Accounting-Game$
```

Figure 2.3: Building the distribution for our new push. /dist has been added in our repo.

The **dist** folder will be renamed **AccountingGame** and placed in **/var/www** as the new version of the game.

NOTE: Before replacing the active version of the game, run `sudo mv AccountingGame <backup name>` from **/var/www** to create a backup of the current version in case something breaks!

Run `sudo mv /home/<auburn username>/Accounting-Game/dist /var/www/AccountingGame` to replace the active version of the folder with your changes.

```
jpl0048@accounting-game:~/Accounting-Game$ ls
LICENSE    deploy_script.sh  index.html         package.json  screenshot.png  tests
Previous   dist              node_modules       populate.py   src             vite.config.js
README.md  docs              package-lock.json  public        style.css
jpl0048@accounting-game:~/Accounting-Game$ sudo mv /home/jpl0048/Accounting-Game/dist /var/www/AccountingGame
jpl0048@accounting-game:~/Accounting-Game$ cd /var/www
jpl0048@accounting-game:/var/www$ l
Accounting-Game/                    AccountingGameBackup-03-20-26/      AccountingGame_backup_2026_03_02/
Accounting-Game-backup/             AccountingGame_backup2_04_19_2026/  AccountingGame_backup_2026_03_02_v2/
Accounting-Game_backup2_04_19_2026/ AccountingGame_backup3_04_19_2026/  AccountingGame_backup_2026_03_03/
Accounting-Game_backup3_04_19_2026/ AccountingGame_backup_04-07-2026/   AccountingGame_backup_2026_03_22/
Accounting-Game_backup4_04_19_2026/ AccountingGame_backup_04_06_2026/   AccountingGame_backup_2026_04_08_16_54/
Accounting-Game_backup_04_14_2026/  AccountingGame_backup_04_12_2026/   AccountingGame_backup_2026_04_19_20_09/
Accounting-Game_backup_04_16_2026/  AccountingGame_backup_04_14_2026/   AccountingGame_backup_old/
Accounting-Game_backup_04_19_2026/  AccountingGame_backup_04_16_2026/   dummybackup/
Accounting-Game_backup_04_20_2026/  AccountingGame_backup_04_19_2026/   html/
Accounting-Game_backup_04_21_2026/  AccountingGame_backup_04_20_2026/   package-lock.json
AccountingGame/                     AccountingGame_backup_2025-09-24/
```

Figure 2.4: Moving the distribution to /var/www/AccountingGame. Our changes are now live!

NOTE: After pushing your changes, you must restart the backend by running `sudo systemctl restart accountinggame-backend` from any location.

Other Tips

Accessing Console Logs

Upon restarting the live server, you can check the status of the server using `sudo systemctl status accountinggame-backend`.

```
● accountinggame-backend.service - Accounting Game FastAPI Backend
     Loaded: loaded (/etc/systemd/system/accountinggame-backend.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-04-23 06:22:41 UTC; 1 week 0 days ago
   Main PID: 409490 (uvicorn)
      Tasks: 5 (limit: 9434)
     Memory: 81.4M (peak: 82.5M)
        CPU: 26min 12.007s
     CGroup: /system.slice/accountinggame-backend.service
             └─409490 /var/www/Accounting-Game/src/backend/venv/bin/python3 /var/www/Accounting-Game/s
```

Figure 2.5: Checking the status of the active server.

NOTE: To listen for live console outputs, you can use `sudo systemctl restart accountinggame-backend`.

Role Overrides

Whether for testing purposes or users with special cases, you can override the role that the SSO assigns a user by entering their User ID into **/public/assets/role_override.xlsx**. This would most commonly be used to assign an Admin or a TA that is recognized as a student.

Admin (User ID)		Professor/TA (User ID)	Section (Year/Semester, Class Number, Section Number)
Example: **abc0123**		Example: **abc0123**	Example: **2026S_ACCT2110_001**
mld0076		sms0184	002
mclelaj		cds0122	001
eac0124		hkg0017	001

Figure 2.6: Populated role_override.xlsx. The example cells will be overlooked by the parser.

Forcing Scenes

If a role override doesn't work, or you need to force a scene for any reason, you can force a scene by **pressing F12**, **navigating to the console tab**, and running `const activeScene = Phaser.Display.Canvas.CanvasPool.pool[0].parent.game.scene.getScenes(true)[0];` followed by `activeScene.scene.start(<scene name>);`.

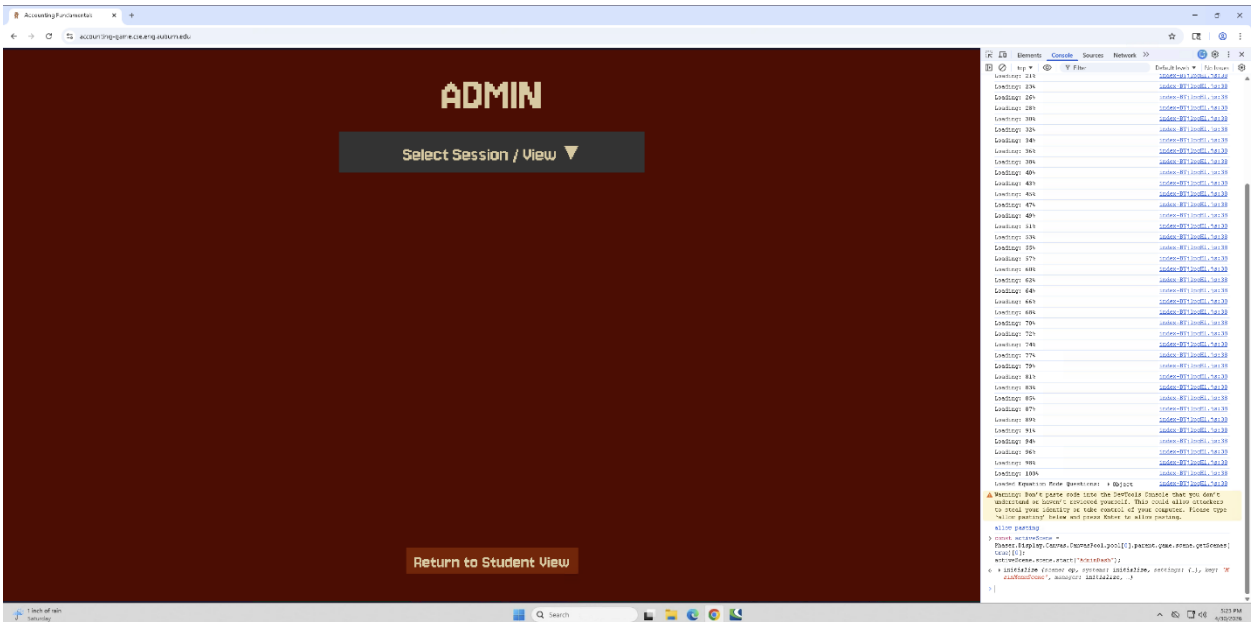

Figure 2.7: Forcing AdminDash from inside an RDP machine located in Shelby.

Additional Information

SSL/HTTPS

The SSL certificate is provided by Auburn IT and lives on the server at:

Certificate: `/etc/certificate/accounting-game_cse_eng_auburn_edu_cert.crt`

Private Key: `/etc/certificate/accounting-game.key`

These are managed by Auburn IT. Please contact Kelly Price if the certificate needs to be renewed.

SAML Authentication

Configuration Files

`/etc/accountinggame/saml/settings.json` | SP and IdP settings

`/etc/accountinggame/saml/advanced_settings.json` | Security settings

SAML_PATH environment variable in .env points to this directory

SP Metadata URL (must be on Auburn wifi/VPN)

https://accounting-game.cse.eng.auburn.edu/saml/metadata

SAML Certificates

The SP certificates are stored outside the project directory on the server at `/etc/accountinggame/saml/certs/` to prevent them from being overwritten by git pulls. They should never be committed to git.

Generating New Certs

In the event that you need to generate new certificates, you must contact Jason Buckman and re-register them as the old certs will stop working. To generate certs use the command:

```
openssl req -new -x509 -days 3652 -nodes -out sp.crt -keyout sp.key
```

Then copy from your local machine using:

```
scp sp.crt <username>@accounting-game.cse.eng.auburn.edu:/tmp/sp.crt

scp sp.key <username>@accounting-game.cse.eng.auburn.edu:/tmp/sp.key
```

Then on the server:

```
sudo mv /tmp/sp.crt /etc/accountinggame/saml/certs/

sudo mv /tmp/sp.key /etc/accountinggame/saml/certs/
```

```
sudo chown -R www-data:www-data /etc/accountinggame/saml/certs

sudo chmod 750 /etc/accountinggame/saml/certs

sudo chmod 640 /etc/accountinggame/saml/certs/sp.key

sudo systemctl restart accountinggame-backend
```

To verify that certs are in place:

```
sudo ls -la /etc/accountinggame/saml/certs/
```

NOTE: All files in `/etc/accountinggame/saml` must be owned by www-data since the backend service runs as that user. If you add or modify any files in that directory, run `sudo chown -R www-data:www-data /etc/accountinggame/saml/` to restore the correct ownership.

AD Group Parser

Upon a user's successful login, Auburn's SSO redirects to the Assertion Consumer Service endpoint via the **/saml/acs** route, which is found in **main.py**. It sends all the user data to the backend as a huge list of attributes. The parser looks through all the Common Names, searching for 3 specific things.

- "CN=AUA_Student_Gids" means this user is a student
- "CN=AU_Teaching" means this user is an instructor
- "CN=2026S_ACCT2110_001" is an example of class membership to Accounting 2110, section 001 in the Spring of 2026

NOTE: If a user is both an instructor and a student, you will not be able to tell which classes they teach and which they do not. Currently, this is handled by giving them a unique role that Admins should manually use to reassign using the role override feature.

This data gets UPSERT into the database and saved to the browser as a signed cookie. Ensure the .env file has the SESSION_SECRET, as this is how Session Middleware signs the cookie.

Database

PostgreSQL database stores the leaderboard and analytics data. To connect to database and be brought into psql on the server run:

```
sudo -u postgres psql -d leaderboard
```

Tables

scores | legacy leaderboard data (archived)

player_profiles | student user information; stores name, Auburn username, and section

game_analytics | game session data; stores game mode, score, and time played.

Schema

See `src/backend/schema.sql` for full schema. Run it with:

```
psql -U postgres -h 127.0.0.1 -d postgres -v password='YOURPASSWORD' -f
schema.sql
```

NOTE: The server database password is different from the local password, contact the previous team for server credentials

NOTE: TRUNCATE permissions must be granted manually to `game_app` on a fresh local setup

NOTE: RESTART_IDENTITY has been removed from the clear data endpoint due to sequence ownership restrictions on `game_app`

Populate.py

If any sort of “dummy data” is needed to test rendering or database insteractions the python script `populate.py` can be used. Run `python3 populate.py` in either local or on the server to automatically recreate three class sections of randomized users. If you design different class names, student names, or scoring metrics this script is very adaptable and can be changed to fit growing needs.

API Endpoints

All backend routes are defined without the /api/ prefix in main.py. nginx strips /api when fowarding requests from the frontend to the backend.

Leaderboard and Scores

/leaderboard/{game} | GET | Get leaderboard for a game

/submit | POST | Submit a score

/preview | GET | Preview rank for a score

User Session

/fetch-user | GET | Returns current logged-in user's session data

/register-profile | POST | Register or update a player profile

Analytics

/stats/section/{id} | GET | Section stats for professor dashboard

/stats/section/{id}/csv | GET | Download section stats as CSV

/stats/admin/global-tops | GET | Global top scores for admin

/stats/admin/global-tops/csv | GET | Download global top scores as CSV

/stats/admin/all-students | GET | All student data for admin dashboard

/api/stats/admin/all-students/csv | GET | Download all student data as CSV

Admin

/admin/clear-data | DELETE | Truncates game_analytics and player_profiles (permanent)

SAML

/saml/metadata | GET | SP metadata; shared with Auburn IT

/saml/login | GET | Initiates SSO login

/saml/acs | POST | Receives SAML response from Auburn

/saml/sls | GET | Handles logout

/saml/logout | GET | Initiates logout

/saml/fake-login | POST | For testing without Auburn SSO

NOTE: Full interactive API docs available at: https://accounting-game.cse.eng.auburn.edu/docs